

APUNTES DE RUST

DECLARACIÓN DE VARIABLES

La declaración de la variable es inversa a c:

Nombre_de_variable: tipo

var1: f32

var2: f64

Los tipos en comparativa:

int	-> i32	char	-> char (son 4bytes unicode no solo ascii)
-----	--------	------	--

long	-> i64	bool	-> bool
------	--------	------	---------

float	-> f32	char*	-> &str
-------	--------	-------	---------

double	-> f64	std::string	-> String
--------	--------	-------------	-----------

U-Int -> u32

U-long -> u64

Para asignar un valor a una variable seria:

```
let a = 32; //no habría una declaracion previa tipo a: i32, y Rust deduce lo que es.\
```

```
let a = 32 as f32 //le indicamos que es un float
```

o

```
let a: i32 = 42;
```

LET es para hacer que no pueda variar (NO PUEDE SER GLOBAL). **MUT** para que varie (Hay que poner **let** delante), y **CONST** para constante (si puede ser global)

```
let mut i = 0; //tipico para hacer while ya que la i variará. No global. En runtime
```

```
const i: i32 = 0; //no puede mutar NUNCA en compilacion y puede ser global. Necesita ser declarado el tipo. y se crea en tiempo de compilación
```

VARIABLES GLOBALES

El tener una variable global en Rust es un problema, ya que puede haber data races para modificarla así que tiene que estar protegida.. si no no deja.

La forma clásica es:

```
static mut A: i32 = 0; //variable global
```

```
fn main(){
```

```
    if true{
```

```
        unsafe{ A = 10;} //no se recomienda pero permitido. unsafe le dice el compilador que confie en el programador.
```

```
    }
```

```
}
```

Unsafe es un peligro.. así que usamos **atomic** para no provocar data races:

```
use std::sync::atomic::{AtomicI32, ordering}
```

```
static A: AtomicI32 = AtomicI32::new(0); //no hace reserva de memoria. Solo inicializa a cero
```

```
fn main() {
```

```
    if true {
```

```
        A.store(10, Ordering::Relaxed); //Relaxed es el nivel más débil del ordenamiento atómico. solo
```

```
me importa modificar el valor, no el orden del tiempo en que se haga.
```

```
    }
```

```
}
```

ESTRUCTURAS

Rust no tiene objetos ni clases, pero sí que puede acercarse a ellos con las estructuras. En Rust todo es privado que se declare. Para hacerlo accesible hay que ponerle la palabra **pub** delante. No solo a la estructura.

(**fichero.rs**)

```
pub struct Prueba{ //debe ser CamelCase P mayúscula
```

```
    pub var1: i32,
```

```
    pub var2: char, //si no son pub estas variables no se puede acceder a ellas
```

```
}
```

Importando el **fichero.rs** como

(fichero: main.rs)

```
mod fichero; //importa el fichero fichero.rs

fn main(){
    let a = fichero::prueba { var1: 42, var2: 'a'};
}
```

Ahora imaginemos que tenemos un archivo.rs que tiene esto:

(fichero.rs)

```
pub struct Prueba{ //para implementar el getter tiene que ser la estructura pub
    x: i32,
}

pub fn test() -> Prueba{ //-> devuelve un tipo Prueba (estructura)
    Prueba {x: 56}
}
```

No habría problema en darle valor dentro de fichero.rs PERO si queremos acceder a el, tendríamos que hacer un metodo de la estructura en el propio archivo fichero.rs:

```
impl Prueba{
    pub fn get_x(&self) -> i32{ //tiene que ser pub tambien. -> i32 = devuelve un int.
        //&self es como los objetos.. o this en c++ va implicito
        (self) lo movería el valor
        (&self) es immutable
        (&mut self) lo podría cambiar.
        self.x //IMPORTANTE!!! SI NO TIENE ';' ES UN RETURN IMPLICITO!!!
    }
}
```

En el main.rs:

```
mod fichero;

fn main (){
    let a = fichero::test(); //llama a la funcion externa en fichero y le da el valor 56
    println!("{}", a.get_x());
}
```